TECHNICAL PROJECT STUDY NOTES

Detecting GAN-Based Data Inversion in Collaborative Deep Learning

Project Walkthrough, Implementation Review, and Interview Preparation

What this document contains

Project motivation and objective; architecture and data flow; explanations of all major code blocks; feature engineering and anomaly detection; evaluation metrics; real-time simulation and reporting; implementation bugs and limitations; and a question bank with interview-ready answers.

Technology stack:	Python, PyTorch, NumPy, scikit-learn, pandas, Matplotlib, Plotly
Core detector:	Isolation Forest (with optional One-Class SVM)
Application setting:	Simulated collaborative-learning client submissions

Prepared as a study guide for technical interviews

Contents

1 Executive Summary **1**
1.1 What is implemented, simulated, and missing? . . . 1
1.2 The main technical caveat . . . 2

2 Project Motivation and Problem Understanding **2**
2.1 What is collaborative or federated learning? . . . 2
2.2 What is a gradient? . . . 2
2.3 What is a gradient-inversion attack? . . . 2
2.4 What this prototype is trying to do . . . 2

3 End-to-End System Architecture **3**
3.1 Major classes and their responsibilities . . . 3

4 Detailed Walkthrough of the Code **3**
4.1 Packages and imports . . . 4
4.2 CNN architecture: `SimpleModel` . . . 4
4.2.1 Tensor dimensions . . . 5
4.3 MNIST loading and normalization . . . 5
4.4 Splitting data among clients . . . 5
4.5 Calculating and collecting normal gradients . . . 5
4.6 GAN architecture and the purported malicious-gradient generation . . . 6
4.6.1 What is wrong in the shown training routine? . . . 6
4.7 Label assignment and saving raw records . . . 7
4.8 Gradient feature extraction . . . 7
4.8.1 Nineteen features per parameter array . . . 7
4.8.2 Why does each sample have 155 features? . . . 8
4.8.3 StandardScaler and the leakage problem . . . 8
4.9 Saving and visualizing the feature matrix . . . 8
4.10 Training the anomaly detector . . . 8
4.10.1 Isolation Forest . . . 8
4.10.2 One-Class SVM option . . . 9
4.10.3 What does contamination mean? . . . 9
4.10.4 Prediction, score, and the threshold . . . 9
4.11 Evaluation metrics . . . 9
4.12 Real-time monitoring wrapper . . . 10
4.12.1 Monitoring statistics and how to interpret them . . . 10
4.13 Monitoring simulation: 5 normal IDs and 2 adversary IDs . . . 10
4.14 Dashboards and logs . . . 11
4.15 Complete system wrapper . . . 11

5 Final Demonstration and Reporting Code **11**
5.1 Normal gradient used in the demo . . . 11
5.2 Synthetic anomaly used in the demo . . . 11
5.3 Three calls in the demo . . . 11
5.4 A bug in the repeated-submission printout . . . 12
5.5 A bug in the summary report's conditional formatting . . . 12
5.6 ZIP archive and file listing . . . 12

6 Files the Notebook Is Intended to Produce **13**

7 Important Implementation Issues and Recommended Fixes **13**
7.1 A more reliable evaluation plan . . . 15

8 Interview Questions and Suggested Answers **15**
8.1 Project overview and motivation . . . 15
8.1.1 Q1. Explain your project in one minute. . . . 15
8.1.2 Q2. What problem are you trying to solve? . . . 15
8.1.3 Q3. What is the difference between collaborative learning and federated learning? . . . 15
8.1.4 Q4. Why is keeping data local not enough for privacy? . . . 16
8.1.5 Q5. What is the main output of your detector? . . . 16
8.2 CNN, gradients, and data . . . 16
8.2.1 Q6. Why did you use MNIST? . . . 16
8.2.2 Q7. Explain the CNN dimensions. . . . 16
8.2.3 Q8. What does backpropagation do? . . . 16
8.2.4 Q9. Why call `optimizer.zero_grad()`? . . . 16
8.2.5 Q10. Why does the model return logits rather than softmax probabilities? . . . 16
8.2.6 Q11. What does the client split actually do? . . . 16
8.3 GANs and gradient inversion . . . 16
8.3.1 Q12. What is a GAN? . . . 16
8.3.2 Q13. What is a gradient-inversion attack? . . . 17
8.3.3 Q14. Did the current code perform a real GAN-based inversion attack? . . . 17
8.3.4 Q15. Why is this limitation important? . . . 17
8.4 Feature engineering . . . 17
8.4.1 Q16. Why extract statistical features instead of feeding the entire gradient into the detector? . . . 17
8.4.2 Q17. Why are there 155 features? . . . 17
8.4.3 Q18. What do L1 and L2 norms tell you? . . . 17
8.4.4 Q19. What does entropy mean in your features? . . . 17
8.4.5 Q20. Why use StandardScaler? . . . 17

8.4.6 Q21. What is data leakage in your pipeline? . . . 18
8.5 Anomaly detection and evaluation . . . 18
8.5.1 Q22. Why did you use Isolation Forest? . . . 18
8.5.2 Q23. What is the purpose of One-Class SVM in the code? . . . 18
8.5.3 Q24. What does `contamination=0.15` mean? . . . 18
8.5.4 Q25. Explain the confusion matrix. . . . 18
8.5.5 Q26. Why are accuracy and F1 not enough? . . . 18
8.5.6 Q27. Why does the code negate anomaly scores for ROC-AUC? . . . 18
8.5.7 Q28. Does the saved threshold control predictions? . . . 18
8.5.8 Q29. Can we claim the stated accuracy validates the system? . . . 19
8.6 Monitoring and software engineering . . . 19
8.6.1 Q30. What happens when a submission is received? . . . 19
8.6.2 Q31. What does “real-time” mean in this project? . . . 19
8.6.3 Q32. Why might the monitoring statistics be misleading? . . . 19
8.6.4 Q33. What happens when the system is initialized again? . . . 19
8.6.5 Q34. Why be careful when using pickle? . . . 19
8.6.6 Q35. What errors did you identify in the final demonstration/reporting code? 19
8.7 Improvements and project ownership . . . 19
8.7.1 Q36. How would you improve the project next? . . . 19
8.7.2 Q37. What is the most important limitation you would tell the interviewer? . 20
8.7.3 Q38. What part of the project is genuinely implemented? . . . 20
8.7.4 Q39. How would you reduce false positives? . . . 20
8.7.5 Q40. Is an anomalous gradient necessarily malicious? . . . 20
8.7.6 Q41. How could you test generalization? . . . 20
8.7.7 Q42. Why are the plots not independent evidence of correctness? . . . 20
8.7.8 Q43. Does this system reconstruct private images? . . . 20
8.7.9 Q44. Why are the training and demonstration gradients not guaranteed to match? . . . 20
8.7.10 Q45. What should you avoid claiming? . . . 21

9 Interview Cheat Sheet **21**
9.1 Key numbers to remember . . . 21
9.2 A quick vocabulary list . . . 21

10 A Strong and Honest Final Project Pitch **22**
10.1 If an interviewer asks about the resume metric . . . 22
10.2 Final checklist before the interview . . . 22

1 Executive Summary

The project is a prototype for monitoring model-gradient submissions in a collaborative-learning setting. Its central idea is to extract statistical features from gradients and use an anomaly detector to flag submissions that differ from the patterns learned from normal gradients.

The code creates a CNN for handwritten-digit classification, calculates ordinary gradients from MNIST batches, creates synthetic suspicious gradient arrays, converts gradient arrays into statistical feature vectors, trains an Isolation Forest on examples labelled normal, and wraps the detector in a simulated client-monitoring workflow. The workflow can accept a submission, reject an anomalous one, mark the client as isolated for the current session, log decisions, and generate reports and visualizations.

One-sentence summary: We built a statistical gradient-anomaly detection prototype with simulated client monitoring. The current code does **not** yet demonstrate reliable detection of genuine GAN-based gradient-inversion attacks because its suspicious gradients are generated randomly rather than produced by a real inversion procedure.

1.1 What is implemented, simulated, and missing?

Area	Actual status in the code
CNN gradient calculation	**Implemented**: a PyTorch CNN computes gradients from MNIST mini-batches.
Client data split	**Implemented**: the dataset is divided into ten subsets, with each subset processed by a separate newly initialized model.
Gradient-inversion attack	**Not implemented**: no image reconstruction from victim gradients is performed.
GAN training	**Not implemented**: a GAN architecture is defined, but the shown code does not correctly train it.
Malicious-gradient generation	**Partial / simulated**: arrays are generated from random and exponential distributions.
Feature extraction	**Implemented**: statistical features are collected from gradient arrays.
Anomaly detection	**Implemented**: Isolation Forest is trained on the samples labelled normal.
Federated learning / aggregation	**Not implemented**: no global model or FedAvg aggregation is implemented.
Client rejection and isolation	**Partial / simulated**: implemented in a local simulation, not connected to a real network or FL server.
Evaluation and dashboards	**Implemented**: metrics and plots are produced, but their evidential value is limited by the synthetic data and preprocessing leakage.
Production deployment	**Not implemented**: the code is not a complete secure, concurrent, networked service.

1.2 The main technical caveat

The labels “normal” and “malicious” are assigned according to which list a sample came from. The suspicious list contains randomly generated arrays, not gradients produced by a GAN-based inversion attack. Therefore, a high score on this dataset would primarily show how well the detector separates the particular normal-gradient records from the synthetic random arrays. It would not establish that the system can detect real privacy attacks.

2 Project Motivation and Problem Understanding

2.1 What is collaborative or federated learning?

In collaborative learning, multiple clients contribute to training a model. In a common federated-learning setup, clients train locally and send model updates or gradients to a coordinating server. The server aggregates updates and distributes an updated model. The attraction is that raw training data can remain on the clients.

However, keeping raw data local does not automatically guarantee privacy. Gradient values can sometimes reveal information about the examples used to calculate them. Gradient-inversion attacks attempt to exploit that information to reconstruct or infer training data.

2.2 What is a gradient?

A gradient describes how the loss changes with respect to a model parameter. For a parameter θ, the gradient is $\nabla_\theta L$. During ordinary training, gradients are calculated by backpropagation and used by an optimizer to update parameters.

In this project, the code saves gradients from a CNN’s parameter tensors and summarizes them using statistical features. The detector then attempts to identify statistical patterns that look unusual compared with the normal samples used to train it.

2.3 What is a gradient-inversion attack?

A gradient-inversion attack uses observed gradients, and sometimes known model information, to infer the data that generated those gradients. In a real attack, an optimization method would try to find a candidate input whose gradients resemble the observed victim gradients. A GAN could be used as part of a reconstruction or image-prior method, but merely defining a generator and discriminator is not enough to implement such an attack.

2.4 What this prototype is trying to do

The intended defense pipeline is:

1. Observe gradient submissions from clients.
2. Turn each submission into a fixed-length vector of statistical features.
3. Use a model trained on normal samples to compute an anomaly decision and score.
4. Reject submissions flagged as anomalous and isolate the associated client in the current session.
5. Record events and summarize system behavior in logs and dashboards.

Important distinction: this is an anomaly-detection approach, not a direct proof of data leakage. An unusual gradient is not necessarily malicious, and a malicious gradient is not guaranteed to look statistically unusual.

3 End-to-End System Architecture

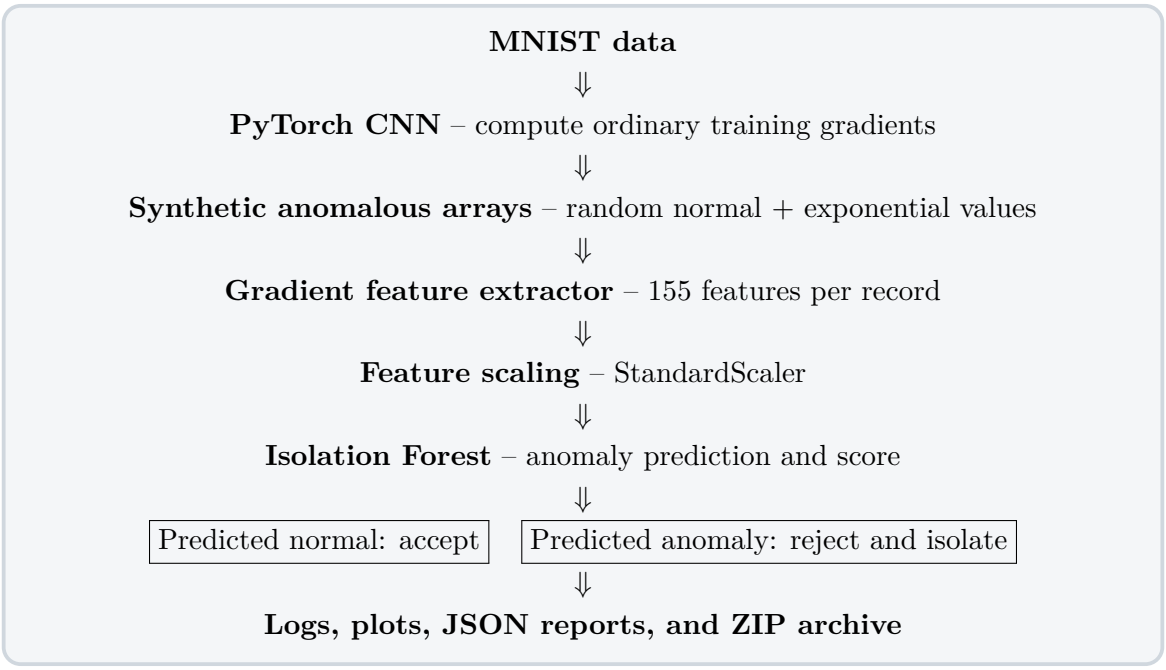


Figure 1: Conceptual flow implemented by the notebook. The anomalous-gradient source is synthetic, not a genuine inversion attack.

3.1 Major classes and their responsibilities

Class / function	Responsibility
`SimpleModel`	Defines the CNN used for MNIST classification and gradient calculation.
`split_dataset_for_clients`	Splits the training dataset into approximately equal subsets.
`train_and_collect_gradients`	Calculates and stores gradients from a limited number of mini-batches per simulated client.
`SimpleGAN`	Defines generator and discriminator networks, but the shown attack-generation routine does not correctly train them.
`generate_malicious_gradients`	Creates records containing synthetic random arrays labelled as GAN inversion.
`GradientFeatureExtractor`	Computes statistical features and scales the feature vectors.
`AnomalyDetector`	Trains an Isolation Forest or One-Class SVM using samples labelled normal and evaluates predictions.
`RealTimeDetectionSystem`	Simulates online submission checking, logging, and client isolation.
`CompleteGANDetectionSystem`	Loads the saved objects and exposes helper methods for detection, status, reports, and saving.

4 Detailed Walkthrough of the Code

4.1 Packages and imports

The setup installs and imports libraries for deep learning, datasets, numerical computation, data frames, plotting, anomaly detection, preprocessing, metrics, serialization, timestamps, and dashboard generation.

- **PyTorch / torchvision:** define the CNN, calculate gradients, load MNIST, and create mini-batches.
- **NumPy / pandas:** manipulate arrays and tabular logs.
- **scikit-learn:** Isolation Forest, One-Class SVM, feature scaling, train/test splitting, and evaluation metrics.
- **Matplotlib / Seaborn / Plotly:** create static plots and an interactive HTML dashboard.
- **pickle / json:** save Python objects and structured reports.
- **datetime:** timestamp events and generated reports.

The notebook uses package-install commands that are typical in Jupyter. The success message printed at the end does not independently guarantee that every installation succeeded. Globally suppressing warnings can also hide useful diagnostics. Several imported packages or metrics are not used by the shown pipeline.

4.2 CNN architecture: `SimpleModel`

The model contains two convolutional layers, max-pooling, two fully connected layers, ReLU activation, and dropout. It is a ten-class image classifier, not the attack detector itself.

```
class SimpleModel(nn.Module):
    def __init__(self):
        super(SimpleModel, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(-1, 64 * 7 * 7)
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

4.2.1 Tensor dimensions

Operation	Output shape for batch size B
Input MNIST batch	$(B, 1, 28, 28)$
First convolution, 32 filters and padding 1	$(B, 32, 28, 28)$
First max-pooling layer	$(B, 32, 14, 14)$
Second convolution, 64 filters and padding 1	$(B, 64, 14, 14)$
Second max-pooling layer	$(B, 64, 7, 7)$
Flatten	$(B, 3136)$
First fully connected layer + ReLU	$(B, 128)$
Dropout	$(B, 128)$
Output layer	$(B, 10)$

The final layer returns ten raw logits. Cross-entropy loss expects these logits and the correct class labels; an explicit softmax is not required before using `nn.CrossEntropyLoss`.

The model has approximately **421,642 trainable parameters**. It extracts gradients from a digit classifier; it does not directly learn to distinguish benign and malicious clients.

4.3 MNIST loading and normalization

The code composes two image transformations:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

MNIST includes 60,000 training images and 10,000 test images. `ToTensor()` converts a grayscale image into a tensor with one channel and scales pixel values from $[0, 255]$ to $[0, 1]$. The normalization operation $(x - 0.5)/0.5$ maps those values approximately to $[-1, 1]$.

The training split is used to generate normal gradient records. The test split is used later in the final demonstration to produce another ordinary batch gradient. Loading or displaying images does not itself train the CNN.

Practical note: code that saves an image into `visualizations/` will fail if the directory does not exist. The same applies to the other output folders used later.

4.4 Splitting data among clients

`split_dataset_for_clients(dataset, num_clients=10)` divides the dataset into ten subsets. It calculates an equal size using integer division and distributes any remainder across the first few subsets. With 60,000 training examples and ten clients, each subset contains 6,000 examples.

The function calls PyTorch's `random_split`. The split is random, but the shown function does not pass an explicit seeded generator, so the exact partition is not guaranteed to be reproducible across runs.

4.5 Calculating and collecting normal gradients

The function `train_and_collect_gradients` initializes a new `SimpleModel`, cross-entropy loss, SGD optimizer with learning rate 0.01 and momentum 0.9, and a mini-batch loader with batch size 32. For every selected mini-batch it:

1. Clears old gradients using `optimizer.zero_grad()`.
2. Performs a forward pass to obtain predictions.
3. Calculates the classification loss.
4. Calls `loss.backward()` to calculate gradients.
5. Copies every available parameter gradient into a NumPy array.
6. Saves the client ID, batch index, epoch index, gradients, and loss.
7. Calls `optimizer.step()` to update the local model.

The condition `if batch_idx >= 10: break` means batch indices 0 through 10 are processed: **11 mini-batches per client**, not ten. With ten clients and one epoch, this produces up to **110 normal gradient records**.

Each client creates its own fresh, randomly initialized model. There is no shared global model, no exchange of updates, and no FedAvg aggregation. This is a simulation of client-specific gradient collection, not a complete federated-learning system. The gradients are captured before the optimizer step and are batch gradients rather than gradients from individual examples.

4.6 GAN architecture and the purported malicious-gradient generation

`SimpleGAN` defines a generator and a discriminator:

- **Generator:** random latent vector of length 100 → 256 units → 512 units → 784 output values, with Tanh at the output.
- **Discriminator:** flattened 784-value image → 512 units → 256 units → one sigmoid score.

The generator can produce a flattened, 28-by-28-sized candidate. The discriminator is intended to score whether an image looks real. The stated architecture alone does not establish that either network has learned anything.

4.6.1 What is wrong in the shown training routine?

- The real MNIST batch is loaded once and reused across iterations.
- The discriminator loss is calculated, but it is not backpropagated and the discriminator optimizer is not stepped.
- The generator loss is backpropagated, but the code does not call `optimizer_g.step()`.
- The target CNN, `target_model`, does not contribute to the GAN loss or computation graph.
- The declared `criterion = nn.MSELoss()` is not used.
- Consequently, the target CNN parameters have no gradients from that GAN loss. The code's fallback branch fills those arrays with random values.

The fallback assignment uses the equivalent idea of:

```
manipulated_grad = np.random.randn(*shape) * 10.0
manipulated_grad += np.random.exponential(2.0, shape)
```

Thus, the returned records are labelled `GAN_inversion`, but the gradient arrays are synthetic random values. No private image is reconstructed, and no genuine gradient inversion is performed.

4.7 Label assignment and saving raw records

Each normal record receives label 0 and each synthetic suspicious record receives label 1. The lists are concatenated and shuffled, and then saved as `data/raw_gradients.pkl`.

With the described loop counts, the expected dataset contains 110 normal records and 100 synthetic suspicious records: 210 records in total. The synthetic class represents roughly 47.6% of this dataset. These are expected counts from the code paths, assuming no earlier errors and that all records are retained.

Labels come from list membership, not an independent verification of actual attack behavior. Pickle files can execute malicious code when loaded; only load pickle files from sources you trust.

4.8 Gradient feature extraction

The `GradientFeatureExtractor` converts a dictionary of gradient arrays into a fixed-size feature vector that can be processed by scikit-learn. Statistical features make differently sized arrays summarizable using the same set of feature types.

4.8.1 Nineteen features per parameter array

No.	Feature	Plain-language meaning
1	Mean	Average value of the gradient entries.
2	Standard deviation	Measures spread around the mean.
3	Variance	Squared measure of spread.
4	Minimum	Smallest gradient value.
5	Maximum	Largest gradient value.
6	Median	Middle value after sorting.
7	25th percentile	Value below which approximately one quarter of the entries fall.
8	75th percentile	Value below which approximately three quarters of the entries fall.
9	Skewness	Describes asymmetry of the value distribution.
10	Excess kurtosis	Describes tail weight relative to a normal distribution.
11	L1 norm	Sum of absolute values.
12	L2 norm	Euclidean magnitude.
13	Infinity norm	Largest absolute value.
14	Sparsity	Fraction of entries whose absolute value is below a near-zero threshold.
15	Histogram entropy	Distributional spread calculated from histogram bins.
16	Range	Maximum minus minimum.
17	Positive count	Number of entries greater than zero.
18	Negative count	Number of entries less than zero.
19	Near-zero count	Number of entries within the code's near-zero threshold.

The entropy feature is computed from a histogram with 50 bins and a Shannon-entropy formula. It measures the distribution of values in a gradient array; it is not itself a measure of private-data leakage.

4.8.2 Why does each sample have 155 features?

The CNN has eight trainable parameter arrays: weight and bias arrays for each of the four parameterized layers (two convolutions and two fully connected layers). The extractor calculates 19 features per array and adds three aggregate features based on layer norms.

$$8 \times 19 + 3 = 155.$$

The three aggregate features are described as the mean, standard deviation, and a maximum-to-minimum ratio of the layer norms. Therefore, the intended feature matrix shape is `(210, 155)` and the label vector shape is `(210,)` when all expected records extract successfully.

The extractor also has a feature-name list, but the described code does not use that list to name output columns. A try/except block skips records that fail extraction, so the number of actual output rows can be lower than the number of input records if extraction errors occur.

4.8.3 StandardScaler and the leakage problem

The extractor's `fit_transform` extracts the features, fits a `StandardScaler`, and returns scaled features plus labels. This is intended to standardize features so that large-magnitude feature dimensions do not automatically dominate the detector.

In the described notebook, `fit_transform(all_gradients)` happens before the train/test split. Therefore, the scaler learns means and standard deviations using both the later training examples and the later test examples. This is data leakage. Correct practice is to split raw examples first, fit feature scaling on training data only, and then transform validation/test data using the fitted scaler.

4.9 Saving and visualizing the feature matrix

The feature matrix and labels are written to:

- `data/features_X.npy`: standardized feature matrix.
- `data/labels_y.npy`: binary labels.
- `models/feature_extractor.pkl`: extractor, including its fitted scaler.

The feature-distribution code plots selected feature indices, including mean, standard deviation, maximum, L1 norm, and entropy. These plots help compare the feature values of the two assigned classes. If the synthetic arrays have very different distributions from genuine gradients, class separation may be easy for reasons that do not generalize to real attacks.

Directories must exist before the code saves files. An unused subplot may be removed when the number of feature plots does not fill the whole grid. A legend must be explicitly requested if one is needed.

4.10 Training the anomaly detector

The `AnomalyDetector` supports Isolation Forest and One-Class SVM. The main training code selects Isolation Forest with contamination set to 0.15.

4.10.1 Isolation Forest

Isolation Forest is an unsupervised anomaly-detection algorithm. It builds randomized trees and uses how easily a sample can be isolated from the rest of the data as a clue that the sample is unusual. A sample that is isolated with relatively short paths tends to be considered more anomalous.

The code configures 100 trees, contamination 0.15, a fixed random seed of 42, parallel execution, and verbose output. It then selects training examples with label 0 and fits the model only on those normal examples. It does not use the malicious labels to train a supervised boundary.

4.10.2 One-Class SVM option

The alternative is a One-Class SVM with an RBF kernel, `gamma='auto'`, and `nu` set from contamination. One-Class SVM attempts to learn a boundary around the examples considered normal. It can model nonlinear boundaries, but performance may be sensitive to feature scaling and parameter settings.

4.10.3 What does contamination mean?

Contamination is the estimated or assumed proportion of anomalies used to set the detector's decision boundary. It does not mean that 15% of submissions are truly malicious, and it is not a direct measure of detector accuracy. The code sets contamination to 0.15 even though the generated dataset has a different assigned suspicious-class proportion.

4.10.4 Prediction, score, and the threshold

Scikit-learn's anomaly estimators typically use prediction value $+1$ for an inlier and -1 for an outlier. The code maps -1 to label 1 (anomaly) and $+1$ to label 0 (normal).

The raw `score_samples` value is treated so that lower values mean more anomalous. For ROC-AUC, the code negates the score to make larger values correspond to the positive anomaly class.

The class also calculates a percentile-based threshold from the training scores. But `predict()` calls the estimator's own `predict()` method, which uses the estimator's internal boundary; it does not use the separately calculated `self.threshold`. A threshold visualization based on that separate value can therefore disagree with the actual predictions.

4.11 Evaluation metrics

The evaluation method calculates predictions, scores, a confusion matrix, classification report, and ROC-AUC. The confusion matrix is unpacked as TN, FP, FN, TP in binary classification.

Metric	Meaning
Accuracy	Fraction of all test records classified correctly.
Precision	Among records predicted anomalous, the fraction that are actually positive according to the dataset labels.
Recall / true positive rate	Among positive-labelled records, the fraction detected.
F1 score	Harmonic mean of precision and recall; useful when balancing missed positives and false alarms.
False positive rate (FPR)	Among negative-labelled records, the fraction incorrectly flagged as anomalous.
ROC-AUC	Measures how well scores rank positive-labelled records above negative-labelled records over possible thresholds.
Confusion matrix	Counts true negatives, false positives, false negatives, and true positives.

The train/test split uses 70% for training and 30% for testing, with stratification and random seed 42. With 210 records, the expected sizes are about 147 train and 63 test records if all samples remain. Stratification preserves the assigned label ratio approximately, but it does not fix data leakage or ensure independence among records created by the same process/client.

The evaluation uses synthetic suspicious arrays and may place similarly generated records into both training and test sets. The scaler is also fitted before the split. Therefore, even strong accuracy, F1, or ROC-AUC values would not substantiate reliable detection of genuine

GAN inversion. They would quantify separation on this particular synthetic dataset under a compromised evaluation procedure.

4.12 Real-time monitoring wrapper

The `RealTimeDetectionSystem` keeps the detector, feature extractor, an event log, a set of isolated client IDs, and a client-history dictionary.

When `process_gradient_submission(client_id, gradients, round_num)` is called, it:

1. Captures a timestamp.
2. Immediately rejects clients already in the isolated-client set.
3. Wraps the incoming gradient dictionary in the format expected by the feature extractor.
4. Extracts and scales the feature vector.
5. Requests an anomaly prediction and score.
6. Adds the decision to the detection log.
7. If anomalous, adds the client ID to the isolated set and returns a rejection response.
8. If predicted normal, adds a record to that client's history and returns an acceptance response.

This is the project's simulated real-time layer. It does not implement a network server, federated aggregator, authentication, secure transport, concurrency control, or durable isolation state.

4.12.1 Monitoring statistics and how to interpret them

- **Total submissions:** number of entries in the detection log.
- **Malicious detected:** number of logged entries flagged anomalous, not necessarily the number of truly malicious submissions.
- **Normal accepted:** total logged entries minus flagged entries, assuming all logged entries have one of those decisions.
- **Isolated clients:** number of client IDs currently stored in the isolation set.
- **Active clients:** number of clients with at least one accepted submission recorded in client history, not necessarily clients currently online.
- **Detection rate:** flagged logged submissions divided by logged submissions. This is the fraction flagged, not recall or a verified true attack-detection rate.

Attempts from already-isolated clients are rejected before they are appended to the regular detection log. Consequently, the statistics can undercount those attempts.

4.13 Monitoring simulation: 5 normal IDs and 2 adversary IDs

The simulation reads raw gradient records and chooses the first 50 normal-labelled records and first 20 suspicious-labelled records. It runs for ten rounds:

- Five normal client IDs submit a record in every round: $5 \times 10 = 50$ attempted normal submissions.
- The two adversary IDs begin submitting at round index 3 and continue through round index 9: $7 \times 2 = 14$ attempted adversary submissions.
- Expected total attempts $= 50 + 14 =$ **64**, provided the lists contain the selected records.

The actual logged count can be lower because a client is not logged again after it has been isolated. The same record-selection lists are cycled through, and the samples are taken from the raw records rather than from an independently held-out test split. These are simulated clients, not real hospitals or network peers.

4.14 Dashboards and logs

The code saves detection logs as JSON and uses pandas/Plotly and Matplotlib to make visualizations.

- **Detection timeline:** plots anomaly scores over rounds. The plot colors clients using the client ID text (for example, whether the ID contains the word "adversary"), not an independent ground-truth attack label.
- **Score distributions:** compares scores for records predicted normal and predicted anomalous. This shows the model's own output groupings.
- **Per-client bars:** shows logged submissions and anomalous decisions by client.
- **Pie charts:** shows the distribution of logged normal/anomaly decisions.
- **Round-level and cumulative plots:** show recorded decisions over time.

These dashboards describe system behavior and predictions. They do not independently verify whether real malicious activity occurred.

4.15 Complete system wrapper

The `CompleteGANDetectionSystem` loads the saved feature extractor and detector from pickle files, builds a new monitoring system, and marks the wrapper initialized. It provides helper methods to detect a gradient, report status, generate a full report/dashboard, and save the models.

The trained detector and fitted scaler can persist via pickle. But initializing the wrapper creates a fresh monitoring object, so logs, client history, and the isolated-client set are reset unless those are separately saved and restored. Loading pickle files is only safe when the files are trusted.

5 Final Demonstration and Reporting Code

5.1 Normal gradient used in the demo

The final block creates a new `SimpleModel`, loads a random mini-batch of 32 examples from `test_dataset`, computes cross-entropy loss, calls backpropagation, and stores each parameter's gradient as a NumPy array. This is a legitimate ordinary classification gradient for that randomly initialized model and batch, but it is not guaranteed to match the distribution of the training-time normal gradients used by the detector.

5.2 Synthetic anomaly used in the demo

The demonstration creates arrays with the same shapes as the CNN parameter tensors using random values scaled by 15 and an exponential component with scale 3. The shape compatibility lets the feature extractor process them. It does not make them the output of an actual GAN inversion attack.

5.3 Three calls in the demo

1. Test a normal gradient submitted by client ID `hospital_test`.
2. Test a synthetic anomalous gradient submitted by `adversary_test`.

3. Submit another gradient for `adversary_test` to see whether previous isolation prevents another attempt.

The code prints the acceptance/rejection status and anomaly score where provided. The prediction is not guaranteed: the detector may flag the normal example or accept the synthetic example.

5.4 A bug in the repeated-submission printout

The code unconditionally accesses `result3['reason']`. If the client was not isolated by the previous call and the third call is accepted, the result dictionary may not contain a `reason` key, leading to a `KeyError`. A safer pattern is:

```
print(f"Result: {'ACCEPTED' if result3['accepted'] else 'REJECTED'}")
if not result3['accepted']:
    print(f"Reason: {result3['reason']}")
```

5.5 A bug in the summary report's conditional formatting

The report contains expressions in the style of:

```
Accuracy: {results['accuracy']*100:.2f}% if 'results' in locals() else 'N/A'
```

Inside an f-string, the conditional expression must be included within the braces. As written in the project, the conditional text is part of the formatted string rather than a condition that is evaluated. This can lead to a report that displays confusing literal text or fails in related accesses if `results` is not defined.

A clearer solution is to build the metric strings first:

```
if 'results' in locals():
    accuracy_text = f"{results['accuracy'] * 100:.2f}%"
    precision_text = f"{results['precision'] * 100:.2f}%"
    recall_text = f"{results['recall'] * 100:.2f}%"
    f1_text = f"{results['f1_score']:.4f}"
    auc_text = f"{results['roc_auc']:.4f}"
else:
    accuracy_text = precision_text = recall_text = "N/A"
    f1_text = auc_text = "N/A"
```

Then insert these already-computed strings into the summary. Also guard the creation of `results/model_performance.json` so it only runs when valid results exist.

5.6 ZIP archive and file listing

The notebook calls `shutil.make_archive('gan_detection_results', 'zip', '.')` to create a ZIP of the current directory. This can include unrelated files or files from previous runs because the source path is the entire current directory. A safer packaging method is to create an explicit staging directory and copy only required artifacts into it before zipping.

The final file walk lists selected file types and sizes. The printed statements that the project is "operational", "complete", or "ready for deployment" are status messages written by the code; they are not independent deployment tests. In particular, the current implementation is a prototype, not a production-secure service.

6 Files the Notebook Is Intended to Produce

The exact set of files depends on which cells ran successfully and whether the required directories existed. The notebook refers to the following outputs.

Path	Purpose
`data/raw_gradients.pkl`	Stored normal and synthetic suspicious gradient records.
`data/features_X.npy`	Feature matrix created from raw records.
`data/labels_y.npy`	Binary labels assigned according to source list.
`models/feature_extractor.pkl`	Saved feature extractor and fitted scaler.
`models/detector.pkl`	Saved anomaly detector object.
`visualizations/mnist_samples.png`	Example MNIST images, if the visualization cell ran.
`visualizations/feature_distributions.png`	Plots of selected statistical features.
`visualizations/roc_curve.png`	ROC curve visualization.
`visualizations/confusion_matrix.png`	Confusion matrix plot.
`visualizations/anomaly_scores.png`	Anomaly-score distribution plot.
`visualizations/interactive_dashboard.html`	Interactive Plotly dashboard.
`visualizations/performance_analysis.png`	Performance-related plot, if created by the relevant cell.
`visualizations/system_dashboard.png`	Summary system dashboard, if created.
`logs/detection_logs.json`	Logged monitoring decisions.
`results/system_config.json`	Configuration and summary metadata.
`results/model_performance.json`	Saved evaluation metrics.
`results/SUMMARY_REPORT.txt`	Plain-text final summary.
`gan_detection_results.zip`	Archive created from the current directory.

The code must create directories such as `data/`, `models/`, `logs/`, `results/`, and `visualizations/` before writing the respective files. Existing files may be overwritten, and the ZIP source should be narrowed to a deliberate list of output files for reliable sharing.

7 Important Implementation Issues and Recommended Fixes

Issue	Why it matters	Recommended correction
Synthetic malicious arrays	A random distribution is not a gradient-inversion attack.	Implement a genuine reconstruction/attack pipeline and collect its real outputs.

Issue	Why it matters	Recommended correction
GAN optimizer steps absent	The generator/discriminator do not learn as intended.	Backpropagate each appropriate loss and step the correct optimizer; verify training behavior.
Target CNN disconnected from GAN loss	Its gradients are absent in the attack-generation routine.	Define an explicit attack objective that uses observed victim gradients and candidate inputs.
No FedAvg/global model	Client training does not simulate real federated model updates.	Add a coordinator, shared model state, local training, and aggregation.
Scaler fitted before split	Test distribution information leaks into preprocessing.	Split first; fit scaler only on the training subset.
Sample-level random split only	Related records from the same generator/client can appear on both sides.	Use client-level, attack-run-level, or scenario-level independent evaluation splits.
Detector only trained on normal records	It learns the normal distribution but not the actual attack class.	Use appropriate normal-only validation or compare with supervised baselines when labels are reliable.
Percentile threshold unused	The displayed threshold may not match actual predictions.	Use one consistent thresholding rule and report the exact decision rule.
Isolation requests not logged	Statistics omit repeat attempts from isolated clients.	Log every attempt, including blocked attempts, with a separate reason/status.
Anomaly treated as malicious	Unusual benign updates may be rejected; stealthy attacks may pass.	Use "anomalous" for the model decision and add validation, appeal, or review logic.
Limited input validation	Malformed arrays, NaNs, wrong shapes, or extreme values can break the pipeline.	Validate layer names, shapes, numeric type, finite values, sizes, and request schema.
Persistence is incomplete	Reinitialization resets logs, history, and isolated clients.	Persist operational state separately and load it reliably.
Pickle loading	Untrusted pickle files can execute code.	Load only trusted artifacts; consider safer serialization and artifact integrity checks.
Summary formatting bug	Conditional text may appear literally or metrics may fail when unavailable.	Compute guarded metric strings before formatting.
ZIP includes whole directory	Unrelated files can be bundled.	Build an explicit staging directory and archive only intended outputs.

7.1 A more reliable evaluation plan

1. Implement at least one genuine gradient-inversion attack and record exactly how the victim gradients and reconstructed inputs are obtained.
2. Generate normal and attack samples under documented settings, including several attack strengths and scenarios.
3. Split data by attack run, client, or scenario so closely related records do not leak between training and test partitions.
4. Split before fitting the scaler. Fit it only on training samples and apply the same fitted scaler to validation and test samples.
5. Use a validation set to choose the anomaly threshold. Keep the final test set untouched until reporting results.
6. Compare Isolation Forest and One-Class SVM with appropriate baselines. Report precision, recall, F1, ROC-AUC, false positive rate, and uncertainty across multiple seeds or runs.
7. Analyze false positives on legitimate client updates and false negatives on real attack examples.
8. Evaluate at the client level as well as the gradient-record level, and document latency, memory use, and throughput if real-time performance is claimed.

8 Interview Questions and Suggested Answers

The answers below are phrased in a way that is technically defensible for the code currently described. They intentionally distinguish implemented behavior from planned or future work.

8.1 Project overview and motivation

8.1.1 Q1. Explain your project in one minute.

I built a prototype for identifying anomalous gradient submissions in a collaborative-learning setting. I used PyTorch to calculate CNN gradients from MNIST, extracted statistical features from the gradient tensors, and trained an Isolation Forest using examples labelled normal. I then built a simulated monitoring layer that scores submissions, rejects flagged updates, isolates clients during the session, and records logs and visualizations. The current version uses synthetic anomalous gradients, so genuine inversion attacks and validation with a real federated-learning system are important next steps.

8.1.2 Q2. What problem are you trying to solve?

Gradient updates can leak information about the training samples that produced them. The prototype attempts to detect gradient submissions whose statistical properties differ from the learned normal pattern, so a monitoring system can flag them for rejection or review.

8.1.3 Q3. What is the difference between collaborative learning and federated learning?

Collaborative learning is a broad description of learning from contributions from multiple parties. Federated learning is a specific family of approaches where clients generally train locally and send model updates to a coordinating server, which aggregates them. The current code simulates client-specific submissions but does not implement the server-side aggregation loop.

8.1.4 Q4. Why is keeping data local not enough for privacy?

Model updates can contain information about the training examples. Some attacks optimize candidate inputs to match observed gradients and thereby infer private training data. Keeping raw files on a client can reduce direct data sharing, but it does not guarantee that updates reveal nothing.

8.1.5 Q5. What is the main output of your detector?

It produces a binary decision derived from the anomaly estimator and a numerical anomaly score. The monitoring wrapper maps the decision to accept or reject and can isolate a client after a flagged submission. The score indicates how unusual a sample appears under the detector; it is not a calibrated probability that the client is malicious.

8.2 CNN, gradients, and data

8.2.1 Q6. Why did you use MNIST?

MNIST is a small, widely used digit-classification dataset, so it makes it easy to build and debug a CNN gradient pipeline. It is convenient for a prototype, but it is not sufficient to establish performance in medical, industrial, or other real collaborative-learning deployments.

8.2.2 Q7. Explain the CNN dimensions.

The input batch has shape $(B, 1, 28, 28)$. The first convolution produces $(B, 32, 28, 28)$ and pooling reduces it to $(B, 32, 14, 14)$. The second convolution produces $(B, 64, 14, 14)$ and pooling reduces it to $(B, 64, 7, 7)$. Flattening gives 3,136 values per image, then the fully connected layers produce 128 values and finally ten logits.

8.2.3 Q8. What does backpropagation do?

Backpropagation applies the chain rule to calculate derivatives of the loss with respect to trainable parameters. In PyTorch, `loss.backward()` populates parameter gradient fields. An optimizer then uses those gradients to update the parameters when `optimizer.step()` is called.

8.2.4 Q9. Why call `optimizer.zero_grad()`?

PyTorch accumulates gradients by default. Clearing old gradients before processing a new batch avoids accidentally adding new batch gradients to gradients from a previous batch.

8.2.5 Q10. Why does the model return logits rather than softmax probabilities?

Cross-entropy loss in PyTorch accepts logits and combines the log-softmax operation with the negative log-likelihood calculation in a numerically stable way. Adding softmax before the loss is unnecessary in this setup.

8.2.6 Q11. What does the client split actually do?

It randomly divides the 60,000 training images into ten subsets, about 6,000 images each. Every subset is processed by a separately initialized model. This simulates separate data ownership but does not create a true federated training process because there is no shared global model or aggregation.

8.3 GANs and gradient inversion

8.3.1 Q12. What is a GAN?

A Generative Adversarial Network has a generator that creates candidate samples and a discriminator that tries to distinguish real samples from generated ones. Training uses competing objectives. A

defined generator and discriminator are not enough; their losses must be backpropagated and their optimizers stepped appropriately.

8.3.2 Q13. What is a gradient-inversion attack?

It attempts to infer or reconstruct a training input by using observed model gradients and an optimization procedure. Typically, candidate inputs are updated so their gradients become similar to the observed victim gradients, potentially using model knowledge or a prior over plausible inputs.

8.3.3 Q14. Did the current code perform a real GAN-based inversion attack?

No. The code defines a simple GAN, but the training routine does not correctly update the generator or discriminator. Also, the target model is not part of the GAN loss computation. The resulting target-gradient arrays are filled by random and exponential noise. I would describe them as synthetic anomalous gradients used to test the pipeline, not as gradients generated by a real inversion attack.

8.3.4 Q15. Why is this limitation important?

A detector can separate artificially generated noise from ordinary gradients and still fail against real attacks. A genuine attack may have subtle or different statistical characteristics, while legitimate clients may also produce unusual gradients. Security claims require realistic attacks, independent test scenarios, and false-positive analysis.

8.4 Feature engineering

8.4.1 Q16. Why extract statistical features instead of feeding the entire gradient into the detector?

Model gradients may contain hundreds of thousands of values. Statistical summarization reduces each sample to a much smaller fixed-length vector and can make classical anomaly detection computationally practical. The trade-off is that summarization can discard structure that may be useful for identifying attacks.

8.4.2 Q17. Why are there 155 features?

There are eight parameter arrays in the CNN. The extractor computes 19 statistics for each, giving $8 \times 19 = 152$, and adds three aggregate norm-based features, giving 155 total features.

8.4.3 Q18. What do L1 and L2 norms tell you?

The L1 norm is the sum of absolute gradient values, while the L2 norm is the square root of the sum of squared values. Both summarize gradient magnitude, but they respond differently to the distribution of values. The infinity norm is the largest absolute entry.

8.4.4 Q19. What does entropy mean in your features?

The implementation calculates Shannon entropy from a histogram of gradient values. It summarizes how spread out the values are across bins. It is a statistical feature, not direct evidence that a gradient contains or leaks private information.

8.4.5 Q20. Why use StandardScaler?

StandardScaler centers each feature using the training mean and scales it using the training standard deviation. This is useful when features have very different scales, particularly for algorithms such as One-Class SVM. The scaler must be fit on training data only.

8.4.6 Q21. What is data leakage in your pipeline?

The code calls the feature extractor's `fit_transform` on the complete dataset before creating the train/test split. That means the scaler uses information from the future test set when calculating its mean and standard deviation. The correct sequence is to split first, fit the scaler on training features only, and then transform the test features without refitting.

8.5 Anomaly detection and evaluation

8.5.1 Q22. Why did you use Isolation Forest?

It is an unsupervised anomaly-detection method that can be trained on examples believed to be normal. It isolates points using randomized trees; points that are easier to isolate tend to be treated as anomalous. It suits a prototype where normal examples are available and attacks may be rare, although its performance depends on feature quality and how representative the normal training data are.

8.5.2 Q23. What is the purpose of One-Class SVM in the code?

It is an alternative normal-only anomaly detector. It learns a decision boundary around the normal examples, using an RBF kernel in the shown implementation. It allows a comparison with Isolation Forest, although the notebook must explicitly train and evaluate both configurations to make a fair comparison.

8.5.3 Q24. What does `contamination=0.15` mean?

It configures an expected or assumed anomaly fraction for Isolation Forest's decision boundary. It does not mean that 15% of clients are really malicious, and it does not guarantee 15% detection accuracy. The threshold should be selected using validation data and the intended operational costs.

8.5.4 Q25. Explain the confusion matrix.

Term	Meaning in anomaly detection
True negative (TN)	Normal-labelled sample correctly predicted normal.
False positive (FP)	Normal-labelled sample incorrectly flagged anomalous.
False negative (FN)	Attack-labelled sample predicted normal.
True positive (TP)	Attack-labelled sample correctly flagged anomalous.

8.5.5 Q26. Why are accuracy and F1 not enough?

Accuracy can hide poor performance when the classes are imbalanced. F1 balances precision and recall but does not directly include true negatives. ROC-AUC assesses ranking over thresholds, while the false positive rate is important because wrongly isolating legitimate clients can interrupt service. Reports should include multiple metrics and inspect errors directly.

8.5.6 Q27. Why does the code negate anomaly scores for ROC-AUC?

In the estimator used here, smaller raw sample scores indicate greater abnormality. ROC-AUC expects higher scores to indicate stronger membership in the positive class. Negating scores reverses the ordering so that larger values correspond to greater anomaly evidence.

8.5.7 Q28. Does the saved threshold control predictions?

Not in the described implementation. The threshold is calculated from a percentile of training scores, but `predict()` uses the estimator's internal prediction boundary. To avoid confusion, the

system should use a single explicit thresholding method consistently for prediction, visualization, and evaluation.

8.5.8 Q29. Can we claim the stated accuracy validates the system?

Not as evidence of real attack detection. The current dataset's suspicious class consists of synthetic random arrays; the scaler is fitted before splitting; and records can share the same data-generation process across train and test. Any reported metrics apply only to that particular setup. Genuine attack data and independent test scenarios are needed before making stronger claims.

8.6 Monitoring and software engineering

8.6.1 Q30. What happens when a submission is received?

The system checks whether the client is already isolated. Otherwise, it extracts and scales the features, requests a prediction and anomaly score, logs the decision, and either accepts the submission or isolates the client if the model predicts an anomaly.

8.6.2 Q31. What does "real-time" mean in this project?

It means that the notebook has an inference function that processes one supplied gradient submission at a time. It does not mean that a continuously running, low-latency production service has been benchmarked or deployed. To support a real-time claim, latency, throughput, concurrent requests, and fault handling should be measured.

8.6.3 Q32. Why might the monitoring statistics be misleading?

The detection-rate field is the proportion of logged submissions flagged anomalous, not recall. Attempts by previously isolated clients are rejected early and are not added to the ordinary detection log, so repeated attempts can be undercounted. "Active clients" means clients with at least one accepted submission in history, not necessarily clients currently online.

8.6.4 Q33. What happens when the system is initialized again?

The saved detector and feature extractor are loaded, but a fresh monitoring system is built. Unless state is explicitly saved and restored, the isolated-client set, client history, and current detection logs restart empty.

8.6.5 Q34. Why be careful when using pickle?

Python pickle is designed to serialize Python objects, but loading an untrusted pickle can execute arbitrary code. The model and extractor files should only be loaded if their source and integrity are trusted.

8.6.6 Q35. What errors did you identify in the final demonstration/reporting code?

The third demonstration call accesses a rejection-only `reason` field even if the result may be accepted. The summary report's conditional metric formatting is written incorrectly, so the conditional text may print literally. The ZIP command archives the entire current directory rather than a carefully selected set of deliverables. Output directories also need to exist before files can be written.

8.7 Improvements and project ownership

8.7.1 Q36. How would you improve the project next?

First, implement and verify a genuine gradient-inversion attack. Second, create realistic normal and attack datasets and split them by client or attack run. Third, remove scaler leakage and

tune thresholds on a validation set. Fourth, add true federated training with a coordinator and aggregation, compare anomaly detectors, and measure false positives and false negatives. Finally, add input validation, persistent logs/isolation state, API security, and latency tests before considering deployment.

8.7.2 Q37. What is the most important limitation you would tell the interviewer?

The most important limitation is that the current "malicious" samples are random synthetic arrays rather than outputs of an actual inversion attack. That limits what the evaluation can prove. It is better to acknowledge this precisely and explain how you would validate the system with a real attack than to overstate the result.

8.7.3 Q38. What part of the project is genuinely implemented?

The CNN gradient extraction, statistical feature engineering, scaling, Isolation Forest training, standard evaluation/plotting, simulated inference wrapper, client isolation logic, event logging, and report generation are implemented in the code described. The GAN inversion itself and full federated-learning integration are not implemented.

8.7.4 Q39. How would you reduce false positives?

Use normal validation data that reflects real client variation, select the threshold based on the cost of rejecting legitimate clients, monitor the false positive rate, and test across clients and data distributions. Instead of permanent isolation on a single anomaly, a production design could use repeated evidence, a quarantine/review path, or a risk score, depending on safety requirements.

8.7.5 Q40. Is an anomalous gradient necessarily malicious?

No. A legitimate client may produce an unusual gradient because of a different data distribution, a rare batch, a training instability, or a model update. Conversely, a malicious gradient may look normal enough to evade the detector. Anomaly detection gives a warning signal, not a proof of malicious intent.

8.7.6 Q41. How could you test generalization?

Hold out complete clients, attack runs, or attack configurations rather than randomly splitting related gradient records. Evaluate against unseen clients and attack strengths. Report variance across multiple random seeds and include legitimate distribution shifts, not only randomly generated anomalies.

8.7.7 Q42. Why are the plots not independent evidence of correctness?

Most plots summarize the same predictions, labels, and scores used by the code. They help understand behavior but do not create new ground truth. For example, coloring a client as adversarial because its ID contains a word is a visualization convention, not independent confirmation of malicious activity.

8.7.8 Q43. Does this system reconstruct private images?

No. The current pipeline does not reconstruct images from gradients. It summarizes gradient values and classifies them as normal or anomalous under an Isolation Forest. Reconstruction would be part of a real attack-generation/evaluation pipeline, not part of the current detector.

8.7.9 Q44. Why are the training and demonstration gradients not guaranteed to match?

The normal training records are collected from separate newly initialized CNNs trained on client subsets. The final demo creates another new model and computes a gradient on a randomly selected batch from the MNIST test set. Those gradients can have different distributions because of initialization, data, and training context.

8.7.10 Q45. What should you avoid claiming?

Avoid claiming verified GAN-based inversion detection, validated real-world accuracy, a complete federated-learning architecture, five malicious training clients, or production deployment unless you have additional code and experiments that substantiate those claims.

9 Interview Cheat Sheet

9.1 Key numbers to remember

Item	Value / explanation
MNIST	60,000 training images and 10,000 test images.
CNN output	10 logits for digit classes 0–9.
CNN parameters	Approximately 421,642 trainable parameters.
Client partitions	10 subsets; approximately 6,000 training samples each.
Gradient batches per client	11 batches with the stated stopping condition.
Expected normal records	10 clients × 11 batches = 110.
Synthetic suspicious records	100, according to the generator loop.
Expected total records	210 if all records are retained.
Features per record	155 = eight arrays × 19 features plus three aggregate features.
Train/test split	70% / 30%, stratified, random seed 42.
Detector setting	Isolation Forest with contamination 0.15 in the main run.
Monitoring simulation	Ten rounds; five normal client IDs each round; two adversary IDs starting at round 3.
Attempted monitoring submissions	Up to 64 (50 normal plus 14 adversary) before accounting for blocked repeat submissions.

9.2 A quick vocabulary list

- **Gradient:** derivative of loss with respect to model parameters.
- **Backpropagation:** calculation of gradients through the model.
- **Gradient inversion:** attempt to infer training data from observed gradients.
- **GAN:** generator-discriminator framework for learning to generate samples.
- **Feature extraction:** transformation from raw gradient arrays to a compact vector of statistics.
- **Anomaly detection:** identification of samples that differ from a learned normal pattern.
- **Isolation Forest:** tree-based anomaly detector based on how readily points can be isolated.
- **Contamination:** assumed anomaly fraction used to define the detector boundary; not a ground-truth attack rate.
- **Data leakage:** use of information from held-out data in training or preprocessing.
- **False positive:** legitimate sample wrongly flagged anomalous.
- **False negative:** attack-labelled sample not detected by the model.
- **ROC-AUC:** threshold-independent ranking metric based on the ROC curve.

10 A Strong and Honest Final Project Pitch

Suggested interview answer:
"My project explores gradient-anomaly detection as a possible defense for collaborative deep learning. I built a PyTorch CNN and extracted gradients from MNIST mini-batches. I then developed a feature extractor that summarizes eight CNN parameter arrays using 19 statistics per array, plus three aggregate features, giving 155 features per sample. I trained an Isolation Forest on samples labelled normal and implemented a simulated monitoring wrapper that scores new submissions, rejects flagged updates, isolates clients for the current session, and records logs and visual reports.

The current version is a proof of concept: its suspicious gradients are generated synthetically, and the code does not yet implement a genuine GAN-based inversion attack or a complete federated-learning server. I also identified an important evaluation issue because the scaler is fitted before the train/test split. My next steps would be to implement a real inversion attack, build independent attack/client test splits, correct the preprocessing order, and evaluate false positives and false negatives before making security claims."

10.1 If an interviewer asks about the resume metric

If your resume says the project achieved over 93% accuracy or an F1 score above 0.90, only repeat those numbers when they are supported by actual saved output from the run you are discussing. Even if those values were obtained, clarify that they come from the current synthetic dataset and should not be interpreted as validated performance against genuine inversion attacks. The experimental design should be corrected before claiming real-world or attack-level effectiveness.

10.2 Final checklist before the interview

- Explain the difference between the intended research problem and the current prototype.
- Know why the CNN is present and why its gradients are collected.
- Be able to explain the 155-feature calculation.
- Explain why Isolation Forest is trained using the normal-labelled training records.
- Distinguish contamination, anomaly score, prediction threshold, recall, F1, and ROC-AUC.
- Acknowledge the synthetic attack data and scaler leakage without hiding them.
- Explain exactly what the monitoring wrapper does and which statistics can be misleading.
- Present a concrete next-step plan: genuine attack generation, correct splitting, validation thresholding, real FL integration, and robust evaluation.

End of project study guide. Use this document as a technical reference; verify actual metric values from your executed notebook before quoting them.